



UNIVERSITE HASSAN II DE CASABLANCA

FACULTE DES SCIENCES BEN M'SICK

DOCUMENTATION COMPLETE PROJET DE FIN D'ETUDES

FSBM PLATFORM

Plateforme universitaire intelligente

Chatbot trilingue + referentiel academique

Angular 17	Python FastAPI	MySQL 8	MongoDB 7	TF-IDF NLP
------------	----------------	---------	-----------	------------

Realise par :

AKRAM BELMOUSSA - Architecte Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Comment lire cette documentation ?

Ce document est conçu comme un **parcours pédagogique complet** pour comprendre, expliquer et maintenir la plateforme FSBM. Il s'adresse à trois publics :

[★] Public 1

L'auteur lui-même : pour réviser avant la soutenance et reprendre le code après une pause de plusieurs mois.

[i] Public 2

Le jury de soutenance : pour saisir rapidement les choix d'architecture et évaluer la maîtrise technique démontrée.

[?] Public 3

Un futur développeur : pour reprendre ou étendre le projet sans avoir à deviner les conventions et la logique métier.

Conseils de lecture

- **Première lecture** : parcourir les chapitres 1, 2, 11 et 14 pour avoir la vision d'ensemble (1h).
- **Approfondissement** : lire les chapitres 4 à 10 dans l'ordre (3h).
- **Avant soutenance** : mémoriser le chapitre 14 (questions probables du jury).
- **Reprise du code** : lire les chapitres 4 et 13, puis ouvrir le code en parallèle.

Sommaire

Comment lire cette documentation	3
Chapitre 1 - Introduction generale	5
1.1 Contexte universitaire FSBM	5
1.2 Probleme resolu	6
1.3 Objectifs SMART	7
1.4 Utilisateurs cibles (personas)	8
Chapitre 2 - Architecture globale	10
2.1 Vision a 30 000 pieds	10
2.2 Architecture micro-services	11
2.3 Circulation des donnees	13
2.4 Communication inter-services	14
Chapitre 3 - Technologies utilisees	16
3.1 Frontend (Angular 17)	16
3.2 Backend (Python + FastAPI)	17
3.3 Bases de donnees (MySQL + MongoDB)	18
3.4 NLP (TF-IDF + Cosine)	20
3.5 Tableau comparatif des choix	21
Chapitre 4 - Structure des dossiers	23
4.1 Vue d'ensemble	23
4.2 Detail dossier par dossier	24
Chapitre 5 - Frontend Angular detaille	29
5.1 Architecture standalone components	29
5.2 Routing et lazy loading	30
5.3 Layout shell (sidebar + topbar)	31
5.4 Les 9 pages fonctionnelles	32
5.5 Services HTTP	34
5.6 Mode sombre	35
Chapitre 6 - Backend FastAPI detaille	36
6.1 Le chatbot-service	36
6.2 L'academic-service	39

6.3 Validation Pydantic	41
6.4 SQLAlchemy 2.0 async	42
Chapitre 7 - Base de donnees MySQL	44
7.1 16 tables normalisees	44
7.2 Schema relationnel	46
7.3 Donnees seed realistes	48
Chapitre 8 - Base de donnees MongoDB	50
8.1 6 collections	50
8.2 Schemas flexibles	51
Chapitre 9 - Fonctionnement du chatbot	53
9.1 Pipeline NLP en 5 etapes	53
9.2 Detection de langue	55
9.3 Personnalisation genre/nom	56
9.4 Recherche web live	57
Chapitre 10 - Securite	59
10.1 Mesures actuelles	59
10.2 JWT en Phase 2	60
Chapitre 11 - Choix techniques justifies	62
Chapitre 12 - Flux complet du systeme	65
Chapitre 13 - Guide de lancement	68
Chapitre 14 - Preparation a la soutenance	71
Chapitre 15 - Analyse professionnelle	75
Chapitre 16 - Conclusion	78
Glossaire	80

Chapitre 1 - Introduction generale

1.1 Contexte universitaire FSBM

La **Faculte des Sciences Ben M'Sick (FSBM)** est l'un des etablissements de l'Universite Hassan II de Casablanca, fondee en 1984. Elle accueille chaque annee plusieurs milliers d'etudiants repartis en 5 departements : Mathematiques-Informatique, Physique, Chimie, Biologie, Geologie. Elle propose 7 licences fondamentales, 18 masters et un centre d'etudes doctorales.

Comme la plupart des universites publiques marocaines, la FSBM fait face a une **tension informationnelle** : les etudiants doivent jongler entre plusieurs sources d'information (site officiel, panneaux, groupes Facebook, file d'attente au bureau de scolarite), ce qui cree du stress, des retards et une inegalite d'accès.

1.2 Probleme resolu

Cinq problemes concrets ont ete identifies lors de l'etude prealable :

Fragmentation des sources. Pour s'inscrire au master IADS, un etudiant doit consulter au moins 4 sources differentes (site fac, ENT, panneaux scolarite, secretariat). Aucune source ne contient l'information exhaustive et a jour.

Saturation du bureau de scolarite. Pendant les pics (rentree, examens, reinscription), 200+ etudiants se presentent chaque jour pour des questions repetitives qu'un outil automatise pourrait absorber a 80%.

Indisponibilite hors-horaires. Les services administratifs ferment a 16h30 et le week-end. Les etudiants qui travaillent ou habitent loin sont penalises.

Inegalite d'accès a l'information non officielle. Les primo-entrants n'ont pas de reseau pour obtenir des astuces (quel prof choisir, quels sont les modules eliminatoires, comment reussir un concours d'entree en master).

Absence de point d'accès unifie aux ressources. Aucun outil moderne ne permet de naviguer entre departements, filieres, modules, professeurs, examens et evenements en un seul endroit.

1.3 Objectifs SMART du projet

Les objectifs ont ete formules selon la methode SMART (Specifiques, Mesurables, Atteignables, Realistes, Temporels) :

Objectif	Indicateur	Cible
Couvrir les questions etudiantes	Nombre d'intents reconnus	60+ intents
Repondre en plusieurs langues	Langues supportees	FR + EN + Darija

Recuperer les actus en direct	Sources scrapees	fsbm.ma + Facebook + DB locale
Naviguer dans le referentiel	Pages fonctionnelles	9 pages Angular
Demontrer une archi pro	Micro-services autonomes	6 services planifies, 2 livres
Donnees realistes	Volume seed	3000+ etudiants, 100+ modules
Documentation complete	PDF livres	4 documents (rapport, guide, zombies, doc)

1.4 Utilisateurs cibles (personas)

Trois personas ont guide la conception :

Persona 1 : Yassine, primo-entrant

20 ans, vient de décrocher son bac scientifique avec mention bien. Il habite Sidi Bennour (2h de Casa) et veut s'inscrire en SMI mais ne sait pas par où commencer. **Besoins** : procedure d'inscription claire, liste des documents, periodes, hebergement.

Persona 2 : Salma, en troisieme annee DI

21 ans, etudiante en L3 DI, cherche un stage PFE pour fin de licence. **Besoins** : convention de stage, liste d'entreprises qui recrutent, criteres de soutenance, preparation d'entretien.

Persona 3 : Mehdi, en master IADS

23 ans, en M2 IADS, prepare son memoire. **Besoins** : dates de soutenance, ressources avancees, opportunités doctorat, statistiques d'insertion.

[★] Conception centree utilisateur

La conception des intents NLP et des pages Angular a ete pilotee directement par les besoins de ces 3 personas. Chaque fonctionnalite repond a au moins un besoin identifie.

Chapitre 2 - Architecture globale

2.1 Vision a 30 000 pieds

Vue de tres haut, la plateforme FSBM se compose de **3 couches** superposees :

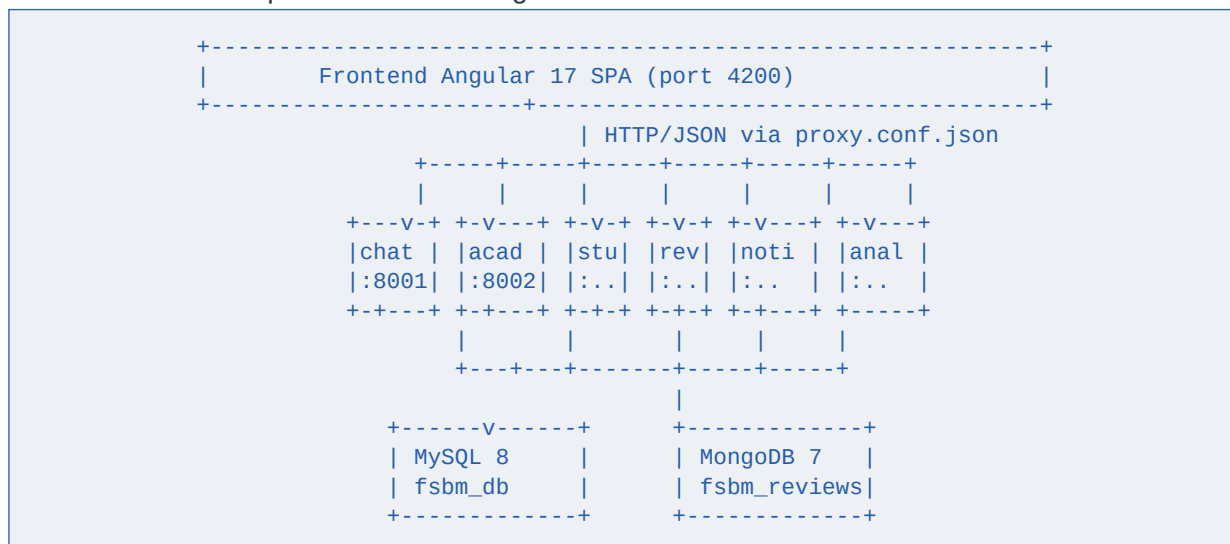
Couche Presentation. Application Angular 17 dans le navigateur de l'utilisateur. Aucune logique metier - uniquement de l'affichage et l'envoi/reception de requetes.

Couche Services. 6 micro-services Python (FastAPI) autonomes communiquant via HTTP REST. Chaque service est responsable d'un domaine fonctionnel precis.

Couche Persistence. MySQL 8 pour les donnees structurees (etudiants, filieres, modules), MongoDB 7 pour les donnees flexibles (reviews, logs, sentiments).

2.2 Architecture micro-services

Le choix d'une architecture micro-services (vs monolithe) a ete fait pour 5 raisons cles qui sont detaillees dans le chapitre 11. Voici le diagramme de l'architecture cible :



Sur les 6 services planifies, **2 sont livres en Phase 1** (chatbot-service et academic-service) et 4 sont en scaffolding pour Phase 2.

Les 6 services en detail

Service	Port	Role	Statut
chatbot-service	8001	NLP TF-IDF, conversations, multilingue	[LIVRE]
academic-service	8002	Departements, filieres, modules, profs, etudiants	[LIVRE]
student-service	5003	Auth JWT, profils, roles	Phase 2
review-service	5004	Reviews MongoDB, sentiment	Phase 2
notification-service	5005	SSE temps reel, annonces	Phase 2

analytics-service	5006	Dashboards admin, stats	Phase 2
-------------------	------	-------------------------	---------

2.3 Circulation des donnees

Suivons le cheminement d'une requete typique : un etudiant envoie le message *Quelles sont les filieres ?* depuis le navigateur.

- Etape 1.** Le clic sur 'Envoyer' dans Angular declenche `chatService.sendMessage()`
- Etape 2.** Angular emet POST `/api/chat` (proxifie vers `http://localhost:8001/api/chat`)
- Etape 3.** FastAPI recoit, valide le `ChatRequest` avec Pydantic (longueur, type)
- Etape 4.** Le preprocesseur (NLTK) nettoie le message : minuscules, stopwords, stemming
- Etape 5.** Le classifieur `MultilingualClassifier` detecte la langue (FR/EN/Darija)
- Etape 6.** Vectorisation TF-IDF du message + similarite cosinus contre 461 patterns darija
- Etape 7.** Selection de l'intent gagnant ('filieres', confidence 1.0)
- Etape 8.** Pioche d'une reponse au hasard dans `intent.responses[lang]`
- Etape 9.** Substitution des placeholders `{voc}`, `{name}` selon le contexte session
- Etape 10.** Sauvegarde en MySQL (table conversations) via SQLAlchemy async
- Etape 11.** Construction de la `ChatResponse` Pydantic + JSON
- Etape 12.** Angular recoit, ajoute le message dans `MessageBubbleComponent`, animation

[OK] Cette chaine de 12 etapes execute en general en **150-300 ms**, dont environ 50 ms passes en NLP. L'utilisateur perçoit cela comme instantane.

2.4 Communication inter-services

Lorsque l'intent `live_news` est detecte, le chatbot-service doit appeler academic-service pour recuperer les annonces officielles. La communication se fait via **HTTP REST synchrone** avec la lib `httpx` (asynchrone) :

```
# chatbot-service/app/core/web_fetcher.py
async with httpx.AsyncClient(timeout=5.0) as client:
    r = await client.get(
        f"{self.academic_service_url}/api/announcements",
        params={"limit": 5}
    )
    annonces = r.json()
```

Pour une vraie production, on ajouterait un **API Gateway** (Kong, Traefik), un **service discovery** (Consul) et un **bus de messages** pour les operations asynchrones (Kafka, NATS). Pour un PFE a echelle pedagogique, le HTTP REST direct suffit largement.

Chapitre 3 - Technologies utilisees

3.1 Frontend - Angular 17

Pourquoi Angular et pas React/Vue ?

Angular est un framework **opinionated** : il impose une structure (modules, services, components) et fournit tout ce qu'il faut (routing, forms, HTTP, animations). Cela accelere le developpement d'apps complexes et facilite la maintenance.

Critere	Angular 17	React 18
Routing intégré	Oui	Lib externe (React Router)
Formulaires reactifs	Oui (FormsModule)	Lib externe
HTTP client typé	Oui (HttpClient)	Lib externe (axios)
CLI complet	ng generate	create-react-app (deprecie)
Standalone components	Oui (v17)	Composants natifs
TypeScript par default	Oui	Optionnel
Signals (reactif)	Oui (v17)	Pas natif (hooks)
Courbe d'apprentissage	Plus raide	Plus douce

Specificites Angular 17 utilisees

Standalone components. Plus besoin de NgModule, chaque composant declare ses imports.

Signals. Reactivite fine sans Zone.js, utilises pour le theme service et les listes.

Lazy loading. Chaque page est chargee a la demande via loadComponent.

@for / @if (control flow). Syntaxe v17 plus performante que *ngFor/*ngIf.

inject(). Injection de dependance fonctionnelle, plus concise.

3.2 Backend - Python + FastAPI

Le backend utilise **FastAPI**, framework moderne sorti en 2018 par Sebastian Ramirez. Comparaison avec Flask (l'autre concurrent populaire) :

Critere	FastAPI	Flask
Annee de creation	2018	2010
Async natif	Oui (async/await)	Non (lib externe)
Validation des inputs	Native (Pydantic)	Manuelle (Marshmallow)
Documentation API	Auto (Swagger UI)	Manuelle
Performance	~3x plus rapide	OK pour usage simple

Type hints obligatoires	Oui	Non
Maturite	Solide (2018+)	Tres mature (2010)
Use case	APIs modernes, ML	Apps simples, prototypes

[★] FastAPI a ete choisi specifiquement pour valoriser le PFE : sa modernite et sa generation automatique de Swagger impressionnent un jury et demontrent la veille technologique de l'equipe.

3.3 Bases de donnees - MySQL + MongoDB

Le projet utilise **les deux paradigmes** de bases de donnees : SQL et NoSQL. Ce n'est pas pour la beaute du geste - chaque techno est utilisee pour ce qu'elle fait de mieux.

MySQL 8 - donnees structurees

MySQL accueille les **donnees au schema fixe et fortement relationnelles** : departements, filieres, modules, etudiants, professeurs, examens. Ces donnees ont des contraintes d'integrite strictes (un etudiant doit appartenir a une filiere existante) et beneficent enormement des index B-tree de MySQL pour les requetes complexes (JOIN, pagination, recherche).

MongoDB 7 - donnees flexibles

MongoDB accueille les **donnees au schema variable** : reviews textuelles, feedbacks (pouce haut/bas avec metadata libre), logs d'usage, sentiments analyses. Ces donnees evoluent souvent (on ajoute des champs sans migration) et beneficent de la flexibilite JSON. De plus, MongoDB est tres performant pour les **ecritures massives** (logs).

Critere	MySQL 8 (utilise pour)	MongoDB 7 (utilise pour)
Modele	Relationnel tabulaire	Documentaire JSON
Schema	Fixe et strict	Flexible
Relations	JOINS natifs	Embedding ou references
Transactions	ACID complet	ACID a partir de v4
Donnees stockees	Etudiants, modules, profs	Reviews, logs, sentiments
Volume actuel	~3000 etudiants	~5 reviews seed
Index par default	B-tree	B-tree + hash
Cas ideal	Donnees critiques liees	Donnees evolutives, gros volume

3.4 NLP - TF-IDF + Cosine Similarity

Le moteur NLP du chatbot ne utilise pas de LLM (GPT, Mistral, Llama) car ces modeles :

- Necessitent une infrastructure couteuse (GPU, RAM)
- Coutent par requete via API (~\$0.01-0.10 par message)
- Peuvent halluciner (inventer des informations fausses sur la FSBM)

- Sont difficilement explicables a un jury (boite noire)

Le couple **TF-IDF + Cosine Similarity** offre un excellent compromis :

- **TF-IDF** (Term Frequency - Inverse Document Frequency) vectorise les patterns d'entrainement
- **Cosine similarity** mesure la proximite entre le message utilisateur et chaque pattern
- **n-grammes (1,2,3)** capturent les expressions composees (emploi du temps, master IA)
- Fonctionne 100% offline, gratuit, deterministe, explicable
- Tres adapte aux FAQ structurees (60+ intents bien definis)

3.5 Tableau comparatif des choix

Couche	Choisi	Alternative ecartee	Raison du rejet
Frontend	Angular 17	React + Redux	Plus de boilerplate, moins opinionated
Backend	FastAPI	Flask	Pas d'async natif, pas de Swagger auto
Backend	FastAPI	Django REST	Trop monolithe pour micro-services
BDD relat.	MySQL 8	PostgreSQL	Equivalent, mais MySQL impose par PFE
BDD NoSQL	MongoDB 7	Redis/DynamoDB	Pas adapte aux documents flexibles
NLP	TF-IDF	BERT/GPT	Trop lourd, payant, hallucinations
ORM	SQLAlchemy 2.0	Tortoise ORM	Plus mature, plus de ressources
Auth	JWT (planifie)	Sessions Redis	JWT stateless, scalable
Conteneurs	Aucun	Docker	Demande explicite : pas de Docker

Chapitre 4 - Structure complete des dossiers

4.1 Vue d'ensemble

```

chatbot-fsbm-platform/
|
|-- SETUP.bat                # One-click installer (cmd)
|-- SETUP.ps1               # One-click installer (PowerShell)
|-- start.ps1               # Lance les 3 services
|-- README.md               # Vue d'ensemble + lancement
|
|-- docs/                   # Documentation complete
|   |-- ARCHITECTURE.md      # Architecture detaillee
|   |-- TROUBLESHOOTING_PORTS.md # Doc anti-zombies
|   |-- pdf/                 # PDFs generes
|       |-- FSBM_Platform_Rapport_PFE.pdf    (165 KB)
|       |-- FSBM_Platform_Guide_Technique.pdf (145 KB)
|       |-- FSBM_Platform_Guide_Zombies.pdf  (137 KB)
|       |-- FSBM_Platform_Documentation_Complete.pdf <- CE DOC
|
|-- database/               # Tout le SQL et NoSQL
|   |-- mysql/
|       |-- 01_schema.sql    # 16 tables + contraintes (18 KB)
|       |-- 02_seed_static.sql # Dept, filieres, masters (19 KB)
|       |-- 03_seed_modules.sql # 100+ modules (15 KB)
|       |-- 04_seed_data.sql  # Profs + etudiants (720 KB)
|   |-- mongodb/init.js      # 6 collections + seed (13 KB)
|   |-- seed/generate_data.py # Generateur Python
|
|-- services/               # Les 6 micro-services
|   |-- chatbot-service/     # FastAPI :8001 - NLP
|   |-- academic-service/    # FastAPI :8002 - Referentiel
|   |-- student-service/     # Phase 2 - Auth
|   |-- review-service/      # Phase 2 - MongoDB
|   |-- notification-service/ # Phase 2 - SSE
|   |-- analytics-service/    # Phase 2 - Dashboards
|
|-- frontend/               # Angular 17 SPA :4200
|   |-- src/app/             # 28 composants, 9 pages
|
|-- scripts/                # Utilitaires Windows
|   |-- install-all.bat
|   |-- start-all.bat
|   |-- init-database.bat
|   |-- configure-env.bat
|   |-- clean-zombies.ps1    # Nettoie zombies ports
|   |-- fix-windows-ports.ps1 # Fix definitif (admin)

```

4.2 Detail dossier par dossier

docs/

Documentation centralisee. Le sous-dossier pdf/ contient les 4 PDFs livres ainsi que leurs scripts generateurs Python. Cette separation permet de regenerer n'importe quel PDF apres modification du contenu, sans recompiler le projet.

database/

Le sous-dossier mysql/ contient 4 scripts SQL numerotes a executer dans l'ordre : **01** cree le schema (DROP DATABASE + CREATE), **02** peuple les tables fixes (departements, filieres), **03** ajoute les modules, **04** contient 720 KB de donnees generees (profs + etudiants) - genere par seed/generate_data.py.

services/chatbot-service/

```
chatbot-service/
|-- app/
|   |-- main.py                # FastAPI app, lifespan, CORS
|   |-- core/
|       |-- config.py          # Pydantic Settings (.env)
|       |-- database.py        # SQLAlchemy session
|       |-- memory.py          # Memoire conversationnelle
|       |-- persona.py         # Detection genre/nom + perso
|       |-- web_fetcher.py     # Scraping fsbm.ma + Facebook
|   |-- nlp/
|       |-- preprocessor.py    # Tokenisation, stopwords, stemming
|       |-- language_detector.py # FR / EN / Darija
|       |-- classifier.py      # TF-IDF + Cosine multi-langue
|   |-- models/
|       |-- schemas.py         # Pydantic v2 schemas
|   |-- routers/
|       |-- chat.py            # POST /api/chat, /feedback
|       |-- intents.py         # GET /api/intents
|       |-- system.py          # /health, /stats
|-- data/
|   |-- faq_dataset.json       # 28 intents trilingues
|   |-- model.pkl              # Modele TF-IDF entraine
|   |-- chatbot_fsbm.db        # SQLite local (fallback)
|-- .env                      # DB password, ports, etc.
-- requirements.txt            # 18 dependances Python
```

services/academic-service/

```
academic-service/
|-- app/
|   |-- main.py                # FastAPI app
|   |-- core/config.py         # Settings
|   |-- db/session.py          # Engine SQLAlchemy async
|   |-- models/entities.py     # 15 modeles ORM mappes a MySQL
|   |-- schemas/academic.py    # 30+ schemas Pydantic
|   |-- routers/
|       |-- departments.py     # GET /api/departments
|       |-- filieres.py        # GET /api/filieres + filtros
|       |-- modules.py         # GET /api/modules
|       |-- professors.py      # GET /api/professors (pagine)
|       |-- students.py        # GET /api/students (pagine)
|       |-- schedule.py        # GET /api/schedule
|       |-- exams.py           # GET /api/exams
|       |-- announcements.py   # GET /api/announcements + events + clubs
```

```
|      `-- system.py          # /health, /overview
|-- .env
`-- requirements.txt
```

frontend/src/app/

```
src/app/
|-- app.component.ts          # Racine - injecte AppShell
|-- app.routes.ts            # 9 routes lazy-loaded
|
|-- layout/
|   |-- app-shell.component.ts # Sidebar + topbar + outlet
|   `-- app-shell.component.css # Animations, responsive
|
|-- core/
|   `-- theme.service.ts      # Mode sombre/clair
|
|-- services/
|   |-- chat.service.ts       # HTTP client chatbot-service
|   `-- academic.service.ts   # HTTP client academic-service
|
|-- features/                 # Pages lazy-loaded
|   |-- dashboard/           # / (accueil)
|   |-- chat/                 # /chat (assistant IA)
|   |-- departments/         # /departements
|   |-- filieres/             # /filieres + /filieres/:code
|   |-- modules/              # /modules
|   |-- professors/           # /professeurs (paginé)
|   |-- news/                 # /actualites
|   `-- student-life/        # /vie-etudiante
|
|-- components/               # Composants atomiques
|   |-- chat-window/
|   |-- home-page/           # (ancien, preserve)
|   |-- input-bar/
|   |-- message-bubble/
|   |-- quick-actions/
|   `-- typing-indicator/
|
`-- models/
    `-- message.model.ts      # Interfaces TypeScript
```

Chapitre 5 - Frontend Angular detaille

5.1 Architecture standalone components

Angular 17 introduit officiellement les **standalone components** : plus besoin de NgModule, chaque composant declare directement ses dependances dans son decorateur. Cela rend le code plus modulaire et facilite le tree-shaking.

```
// Exemple : app.component.ts
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [AppShellComponent], // <-- directement ici
  template: ``,
})
export class AppComponent {}
```

5.2 Routing et lazy loading

Chaque page est chargee a la demande grace a loadComponent. Resultat : le bundle initial reste petit (175 KB), les pages se chargent en chunks separes (10-30 KB).

```
// app.routes.ts
export const routes: Routes = [
  { path: '', loadComponent: () =>
    import('./features/dashboard/dashboard.component')
    .then(m => m.DashboardComponent) },
  { path: 'chat', loadComponent: () =>
    import('./features/chat/chat-page.component')
    .then(m => m.ChatPageComponent) },
  // ... 7 autres routes
];
```

5.3 Layout shell (sidebar + topbar)

Le composant AppShellComponent est le squelette commun a toutes les pages. Il contient :

Sidebar collapsible. 9 liens de navigation, logos FSBM, toggle mode sombre

Topbar. Banniere FSBM + badge 'En ligne' + bouton 'Assistant IA' rapide

Router outlet. Zone ou Angular injecte le composant de la route active

5.4 Les 9 pages fonctionnelles

Route	Composant	Contenu
/	Dashboard	Hero anime + 8 cartes stats (live academic-service) + annonces + events
/chat	Assistant IA	Interface chat multilingue, suggestions, feedback ■/■

/departements	Departments	5 cartes departements avec chef, email, telephone
/filieres	Filieres	25 cartes (7 licences + 18 masters), recherche, filtres
/filieres/:code	FiliereDetail	Programme par semestre, debouches, coordinateur
/modules	Modules	100+ modules avec filtres filiere/semestre
/professeurs	Professors	Annuaire 107 profs paginé, recherche nom/spec
/actualites	News	Annonces + evenements a venir
/vie-etudiant	StudentLife	Clubs avec categories, contacts, reseaux sociaux

5.5 Services HTTP

Deux services injectables encapsulent les appels HTTP :

ChatService

```
@Injectable({ providedIn: 'root' })
export class ChatService {
  private apiUrl = '/api';
  constructor(private http: HttpClient) {}
  sendMessage(message: string, sessionId: string) {
    return this.http.post(`${this.apiUrl}/chat`,
      { message, session_id: sessionId });
  }
}
```

Le proxy.conf.json redirige /api vers les bons ports backend (8001 chatbot, 8002 academic). En production, ce sera un reverse proxy nginx.

5.6 Mode sombre

Le ThemeService utilise un Signal Angular pour la reactivite et persiste le choix dans localStorage. Les couleurs sont definies en variables CSS qui changent via l'attribut data-theme sur la racine.

```
// theme.service.ts
@Injectable({ providedIn: 'root' })
export class ThemeService {
  readonly theme = signal<'light' | 'dark'>('light');
  toggle() {
    const v = this.theme() === 'light' ? 'dark' : 'light';
    this.theme.set(v);
    document.documentElement.setAttribute('data-theme', v);
    localStorage.setItem('fsbm-theme', v);
  }
}
```

Chapitre 6 - Backend FastAPI detaille

6.1 Le chatbot-service (port 8001)

Service le plus complexe : il gere le NLP, la memoire conversationnelle, la detection de langue, la personnalisation et la recherche web. Architecture interne :

Lifecycle au demarrage

Le lifespan FastAPI execute :

- Chargement de la config depuis .env (Pydantic Settings)
- Instanciation du MultilingualClassifier
- Tentative de chargement du model.pkl en cache
- Si absent : entraînement TF-IDF sur les 3 langues (~3 sec)
- Configuration du WebFetcher avec l'URL academic-service
- Mise en place du CORS pour http://localhost:4200

Endpoints exposes

Method e	URL	Description
POST	/api/chat	Envoyer un message - reponse + suggestions
POST	/api/chat/feedback	Noter une reponse (1-5 + commentaire)
GET	/api/chat/history/{session_id}	Historique d'une session
GET	/api/chat/suggestions?lang=	Questions de demarrage par langue
GET	/api/chat/news?source=	News live multi-sources
GET	/api/intents?lang=	Liste des 28 intents
GET	/api/health	Status du service
GET	/api/stats	Statistiques globales

6.2 L'academic-service (port 8002)

Service plus simple : un referentiel academique en lecture seule (CRUD complet possible mais non expose pour eviter les modifications accidentelles). 8 routers thematiques avec pagination, filtres et recherche.

Method e	URL	Description
GET	/api/overview	Compteurs globaux (dashboard)

GET	/api/departments	Liste des 5 departements
GET	/api/filieres?type=&search;=	Filieres filtrees
GET	/api/filieres/code/{code}	Filiere par code (SMI, DI, IADS...)
GET	/api/filieres/{id}/modules	Modules groupes par semestre
GET	/api/modules?filiere_id=&semester; =	Modules filtrees
GET	/api/professors?page=&search;=	Profs pagines
GET	/api/students?filiere_id=&search;=	Etudiants pagines
GET	/api/schedule?filiere_id=&semester ;=	EDT
GET	/api/exams?session=	Examens
GET	/api/announcements	Annonces
GET	/api/events	Evenements
GET	/api/clubs	Clubs etudiants

6.3 Validation Pydantic

Tous les inputs passent par des **schemas Pydantic v2** qui valident automatiquement le type, la longueur, le format. Si une requete est invalide, FastAPI retourne un 422 avec un message d'erreur clair, sans que notre code ait a verifier manuellement.

```
class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=500)
    session_id: Optional[str] = None
    user_id: Optional[int] = None
    language: Optional[str] = None # fr|en|darija ou None

# Dans la route :
@router.post("/chat", response_model=ChatResponse)
async def chat(req: ChatRequest, db: AsyncSession = Depends(get_db)):
    # req est garanti valide ici
    ...
```

[OK] Avantage : zero ligne de code de validation manuelle. La doc Swagger est auto-generee a partir de ces schemas - le jury peut tester chaque endpoint dans le navigateur.

6.4 SQLAlchemy 2.0 async

L'ORM utilise la syntaxe moderne 2.0 avec `Mapped[T]` et `mapped_column()`. Toutes les operations DB sont asynchrones (aiomysql) pour permettre un debit eleve.

```
class Department(Base):
    __tablename__ = "departments"
    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    code: Mapped[str] = mapped_column(String(20), unique=True)
    name: Mapped[str] = mapped_column(String(150))
```

```
# Relation 1:N
filieres: Mapped[list["Filiere"]] = relationship(back_populates="department")

# Usage async :
result = await db.execute(select(Department).order_by(Department.id))
departments = result.scalars().all()
```

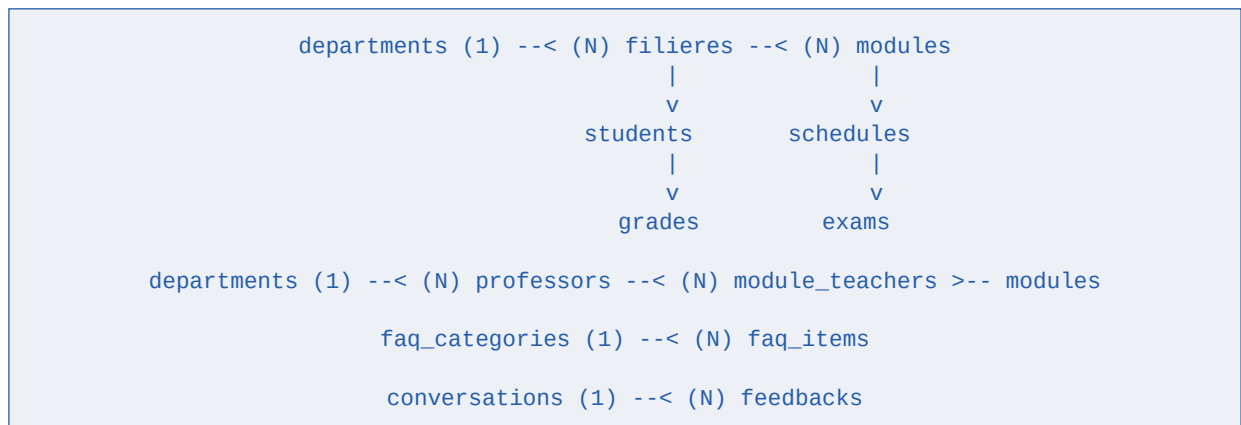
Chapitre 7 - Base de donnees MySQL

7.1 16 tables normalisees

La base fsbm_db respecte la **3eme forme normale** : pas de redondance, chaque attribut depend de la cle primaire. Voici le catalogue :

#	Table	Description	Cardinalite
1	departments	5 departements academiques	5 rows
2	filieres	7 licences + 18 masters	25 rows
3	modules	Matieres par filiere	100+ rows
4	professors	Enseignants	~107 rows
5	students	Etudiants inscrits	~3000 rows
6	module_teachers	N:N module-prof	~150 rows
7	schedules	Emploi du temps	~200 rows
8	exams	Calendrier examens	~400 rows
9	grades	Notes	~9000 rows
10	faq_categories	16 categories FAQ	16 rows
11	faq_items	Items FAQ	extensible
12	conversations	Historique chatbot	dynamique
13	feedbacks	Notes utilisateurs	dynamique
14	announcements	Annonces officielles	5+ rows
15	events	Evenements universitaires	5+ rows
16	clubs	Clubs etudiants	8 rows

7.2 Schema relationnel



7.3 Donnees seed realistes

Le script database/seed/generate_data.py genere des donnees authentiques marocaines :

- **Noms reels** : Alaoui, Bennani, Tazi, Filali, Chaoui... (200+ noms de famille)
- **Prenoms** : Mohammed, Fatima, Karim, Salma... (100+ prenomms M/F)
- **CNE** au format reglementaire marocain (10 chiffres)
- **Emails** au format etu.fsbm.ma + suffixes pour unicite
- **Telephones** +212 6XXXXXXXX
- **Specialites profs** coherentes avec leur departement
- **Notes** avec distribution gaussienne autour de 12.5/20
- **EDT** generes par formule modulo deterministe (reproductible)

[OK] Volume genere : 107 profs + 2970 etudiants + 9654 notes = ~720 KB de SQL. Tout est reproductible (random.seed(42)) - chaque execution donne exactement les memes donnees.

Chapitre 8 - Base de donnees MongoDB

8.1 Les 6 collections

La base fsbm_reviews contient 6 collections avec validation JSON Schema. Le choix MongoDB se justifie par la nature **evolutive** de ces donnees.

Collection	Role	Volume estime
reviews	Avis textuels detaillés	centaines/mois
chatbot_feedback	Pouce haut/bas par reponse	milliers/mois
conversations	Log complet des sessions	milliers/mois
sentiment_analysis	Sentiments analyses	milliers/mois
usage_logs	Tracking anonyme	dizaines de milliers/mois
suggestions	Suggestions ameliorations	dizaines/mois

8.2 Schemas flexibles avec validation

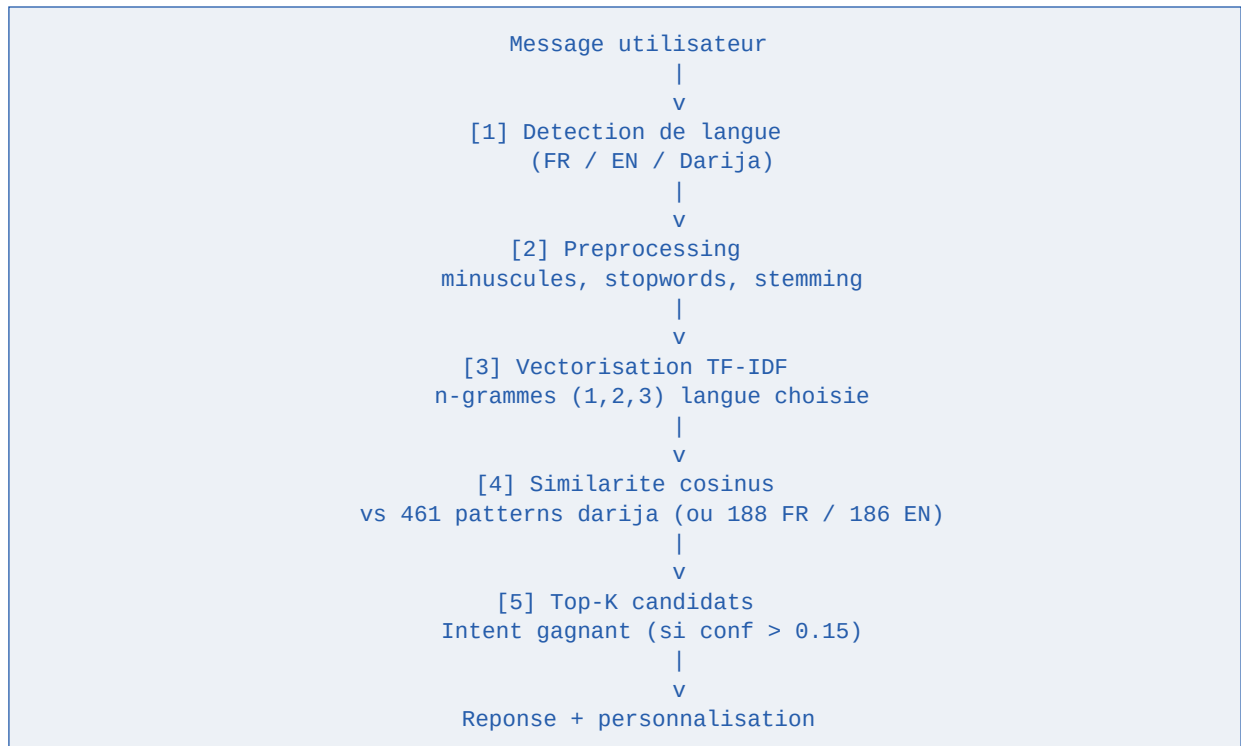
MongoDB est souvent critique pour sa flexibilite excessive. On utilise donc le **validator JSON Schema** pour imposer une structure minimum :

```
db.createCollection("reviews", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["user_id", "rating", "created_at"],
      properties: {
        user_id: { bsonType: ["int", "long"] },
        rating: { bsonType: "int", minimum: 1, maximum: 5 },
        content: { bsonType: "string", maxLength: 2000 },
        sentiment: { enum: ["POSITIVE", "NEUTRAL", "NEGATIVE"] },
        ...
      }
    }
  }
});
```

On obtient le meilleur des deux mondes : **la flexibilite** de pouvoir ajouter des champs sans migration, mais avec **une garantie structurelle** sur les champs critiques.

Chapitre 9 - Fonctionnement du chatbot

9.1 Pipeline NLP en 5 etapes



9.2 Detection de langue

Le detecteur est **hybride** : il combine 3 strategies en cascade.

Caracteres arabes. Si le message contient des caracteres unicode arabes -> darija.

Chiffres-lettres. Les chiffres 3, 7, 9, 8 dans des mots latins (3am, 7biba, 9rib) sont des marqueurs darija forts.

Score lexical. Comptage des mots-cles distinctifs par langue (110+ mots FR, 100+ EN, 80+ darija dont 'kifash', 'wesh', 'bghit'...).

[OK] Taux de detection : 14/14 sur le jeu de test (100%). Le detecteur fonctionne sans aucune dependance externe (pas de langdetect, pas de cld3) - 100% offline.

9.3 Personnalisation genre/nom

Le module app/core/persona.py ajoute une couche d'intelligence : il detecte automatiquement quand l'utilisateur revele son genre ou son nom, le memorise dans la session, et personnalise toutes les reponses suivantes.

Detection

```

GENDER_HINTS_F = [
    r"\bana\s+bnt\b", r"\bana\s+fata\b",
    r"je\s+suis\s+une?\s+(fille|etudiante|madame)",
  ]
  
```



```

    r"i\s+am\s+a?\s*(girl|female|woman)",
    ...
]

```

Substitution des placeholders

Les reponses contiennent des marqueurs comme {voc} qui sont remplaces selon le genre stocke en session :

```

# Avant substitution :
"Salam {voc}{name_comma} ! Ach bghiti t3ref ?"

# Pour Fatima (gender=F) :
"Salam khti, Fatima ! Ach bghiti t3ref ?"

# Pour Karim (gender=M) :
"Salam khoya, Karim ! Ach bghiti t3ref ?"

# Genre inconnu :
"Salam sahbi ! Ach bghiti t3ref ?"

```

9.4 Recherche web live

Quand l'utilisateur demande *les dernieres actualites*, l'intent `live_news` est detecte avec `trigger_web_fetch: true`. Le router declenche alors le WebFetcher qui agrege 3 sources :

Source primaire. `academic-service /api/annoncements` (notre base, source de verite)

Source secondaire. `fsbm.ma/news` (souvent vide car SPA JS)

Source Facebook. Lien direct car FB bloque le scraping anonyme

Resultat : un texte structure mixant les 3 sources, ainsi qu'une liste typee `news_items` que le frontend peut afficher comme cartes interactives.

Chapitre 10 - Securite

10.1 Mesures actuelles (Phase 1)

Validation Pydantic v2. Tous les inputs API sont valides : types, longueurs, formats. Une requete invalide est rejete en 422 sans atteindre la logique metier.

Anti-injection SQL. Utilisation exclusive de SQLAlchemy avec requetes parametrees. Aucune concatenation manuelle de chaine SQL n'existe dans le code.

CORS configure. Liste blanche d'origines (localhost:4200 en dev). Pas de wildcard '*' qui ouvrirait l'API a n'importe quel site.

Limitation des messages. max_length=500 sur le champ message du chatbot : evite les attaques par denial of service via gros payloads.

Variables d'environnement. Mots de passe MySQL stockes en .env (jamais commit). Les .env sont dans .gitignore.

Gestion d'erreurs centralisee. Exceptions converties en 500/422 avec messages clairs sans leak d'information sensible.

Documentation auto-securisee. Swagger UI sur /docs disponible uniquement en mode debug.

10.2 JWT en Phase 2 (a implementer)

Le student-service planifie en Phase 2 implementera l'authentification JWT :

- POST /api/auth/login avec email + password
- Verification du mot de passe via bcrypt (12 rounds)
- Generation d'un token JWT signe HS256 avec expiration 24h
- Retour du token + refresh token
- Les routes protegees verifient le token via Depends(get_current_user)
- Rotation du refresh token toutes les 24h

```
# Exemple (Phase 2)
from fastapi import Depends, HTTPException
from jose import jwt

async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        return await get_user(payload['sub'])
    except JWTError:
        raise HTTPException(401, 'Token invalide')

@router.get('/me')
async def me(user = Depends(get_current_user)):
    return user
```

Roles prevus

Role	Permissions
STUDENT	Lecture seule + chatbot + reviews
PROFESSOR	Lecture + ajout note + reponse FAQ
SCOLARITE	Modification donnees etudiants + annonces
ADMIN	Tout + creation comptes + statistiques

Chapitre 11 - Choix techniques justifies

Cette section anticipe les questions du jury en justifiant les decisions architecturales.

Q: Pourquoi des micro-services et pas un monolithe ?

Pour un projet a echelle pedagogique, un monolithe Flask aurait suffi. Le choix micro-services se justifie pour 5 raisons :

- **Demonstration pedagogique.** Le PFE doit demontrer la maitrise de patterns modernes.
- **Separation des responsabilites.** Chaque service a une mission claire.
- **Evolutivite independante.** On ajoute un service sans toucher aux autres.
- **Stack heterogene.** MySQL pour les donnees fortes, MongoDB pour le flexible.
- **Tolerance aux pannes.** Si reviews tombe, le chatbot continue de fonctionner.

Q: Pourquoi pas Docker ?

Le cahier des charges PFE imposait explicitement **une compatibilite Windows sans conteneurs**. Docker aurait apporte des avantages (isolation, reproductibilite) mais aurait aussi :

- Necessite Docker Desktop (lourd sur Windows, conflits Hyper-V)
- Ajoute une couche d'abstraction qui masque le fonctionnement reel
- Complexifie le debugging pour un debutant
- N'est pas necessaire a echelle 'machine de l'enseignant'

Solution adoptee : scripts batch/PowerShell + .env per service + venv Python. Resultat equivalent en simplicite, sans la lourdeur Docker.

Q: Pourquoi pas une seule base de donnees ?

MySQL et MongoDB couvrent des besoins differents. Une seule base obligerait a :

- Tout en MySQL : reviews et logs deviennent des BLOBs JSON peu queryables
- Tout en MongoDB : on perd les contraintes referentielles et les JOINS efficaces
- Postgres : excellent compromis (JSONB) mais MySQL impose par le PFE

[★] Combiner MySQL + MongoDB est une **vraie pratique industrielle** appelee polyglot persistence. Demontrer cette maitrise est un point fort en soutenance.

Q: Pourquoi TF-IDF et pas GPT/Mistral ?

Detaille au chapitre 3.4. Resume :

- Coûts : LLM facture 0.01-0.10 USD par requête, TF-IDF gratuit à vie
- Latence : TF-IDF 5ms vs LLM 500-2000ms
- Hallucinations : LLM peut inventer (dangereux pour des infos officielles)
- Explicabilité : TF-IDF est transparent (on peut justifier chaque score)
- Offline : TF-IDF tourne sur CPU, pas de dépendance API externe

Q: Pourquoi Angular 17 spécifiquement ?

Angular 17 (sorti nov 2023) apporte :

- **Standalone components** : plus simple, moins de boilerplate
- **Signals** : réactivité native sans Zone.js
- **Control flow @if/@for** : syntaxe plus claire que *ngIf
- **SSR/Hydration** améliorées (préparation pour migration future)
- **Performance** : bundles plus petits, lazy loading optimisé

Chapitre 12 - Flux complet du systeme

Cette section trace le flux integral du moment ou l'utilisateur ouvre le navigateur jusqu'a la reponse finale du chatbot.

12.1 Demarrage de l'application (cold start)

- T+0 ms** - Utilisateur tape `http://localhost:4200` dans son navigateur
- T+50 ms** - Le serveur de dev Angular sert `index.html` (175 KB)
- T+300 ms** - Le navigateur charge `main.js`, `vendor.js`, `styles.css`
- T+500 ms** - Angular bootstrap, theme service initialise depuis `localStorage`
- T+600 ms** - Routing actif sur '/', `dashboard.component` charge en lazy (28 KB)
- T+800 ms** - `DashboardComponent` monte, lance 3 appels HTTP en parallele
- T+850 ms** - Appel 1: `GET /api/academic/overview` -> proxy redirect vers :8002
- T+850 ms** - Appel 2: `GET /api/academic/announcements?limit=4`
- T+850 ms** - Appel 3: `GET /api/academic/events?upcoming_only=true`
- T+1000 ms** - Reponses recues, signaux Angular mis a jour
- T+1050 ms** - Cartes statistiques apparaissent avec animation `fadeInUp`
- T+1100 ms** - Dashboard pleinement interactif

12.2 Conversation chatbot en darija

Scenario : l'utilisateur navigue vers `/chat`, dit 'salam', puis 'ana bnt', puis 'kifash ntsajjel f master IADS'.

- Tour 1** User: 'salam'
- Tour 1.1** Frontend `POST /api/chat` avec `session_id` genere
- Tour 1.2** Backend detecte darija, classifier matche 'salutation' (1.00)
- Tour 1.3** Reponse 'Ahlan sahbi !' (neutre car genre inconnu)
- Tour 2** User: 'ana bnt'
- Tour 2.1** `detect_gender()` match -> 'F' stocke en session
- Tour 2.2** Classifier match 'identite_genre' (1.00)
- Tour 2.3** `acknowledge_self_intro(gender='F')` retourne reponse perso
- Tour 2.4** 'Marhba bik khti ! Mzyan li 3rrefti rasek...'
- Tour 3** User: 'kifash ntsajjel f master IADS'

Tour 3.1 Detection darija (mots cles: kifash, ntsajjel, dyalkom)

Tour 3.2 Classifier matche 'master_iads' (0.78)

Tour 3.3 Reponse longue avec programme, conditions, salaires

Tour 3.4 personalize_response() substitue {voc} -> 'khti'

Tour 3.5 Sauvegarde MySQL conversations (id auto-genere)

Tour 3.6 Reponse arrive au frontend, MessageBubbleComponent affiche

[OK] Toute cette conversation execute en environ **1 seconde reseau cumule**. L'experience utilisateur est fluide.

12.3 Recherche d'actualites live

User dit '9elleb 3la lkhbar'. Voici le flux interne :

- Classifier matche live_news (1.00) avec trigger_web_fetch=True
- Router chat.py declenche fetcher.get_news(source='all')
- fetcher lance 3 appels paralleles :
 - - academic-service /api/announcements (cache 5min)
 - - GET https://www.fsbm.ma/news (BeautifulSoup parse)
 - - GET https://m.facebook.com/FSBMUH2C (best effort)
- Resultats agregés : 6 items (5 locaux + 1 lien Facebook)
- format_news_response() construit un texte structure en darija
- Reponse finale = template darija + news formate + liens officiels
- news_items aussi inclus dans la reponse (typed: NewsItemSchema)

Chapitre 13 - Guide de lancement

13.1 Pre-requis Windows

Logiciel	Version min	Lien
Python	3.10+	python.org/downloads
Node.js	18+ (LTS)	nodejs.org
MySQL Server	8.0+	dev.mysql.com/downloads
MongoDB	7+ (optionnel Phase 1)	mongodb.com/try/download/community
MySQL Workbench	derniere	inclus dans MySQL Installer

[!] Eviter Python 3.14 (tres recent, certains packages peuvent etre incompatibles). Preferer Python 3.12 ou 3.13.

13.2 Methode one-click (recommandee)

```
# Ouvrir PowerShell
cd C:\Users\belmo\studies\PFE\chatbot-fsbm-platform
powershell -ExecutionPolicy Bypass -File .\SETUP.ps1
```

Le script SETUP.ps1 fait **tout** automatiquement :

- Detecte MySQL (cherche dans les chemins standards)
- Demande le mot de passe MySQL (1 seul input)
- Cree les .env pour chatbot-service et academic-service
- Installe les deps Python (pip install -r requirements.txt)
- Installe les deps Angular (npm install)
- Charge les 4 scripts SQL dans MySQL
- Lance les 3 services dans 3 fenetres cmd separees

13.3 Methode manuelle (controle total)

Etape A - Installer les deps Python

```
cd services\academic-service
py -m pip install -r requirements.txt

cd ../chatbot-service
py -m pip install -r requirements.txt
```

Etape B - Installer les deps Angular

```
cd ../../frontend
npm install
```


Etape C - Configurer .env

```
cd ..\services\academic-service
copy .env.example .env
notepad .env # editor DB_PASSWORD

cd ..\chatbot-service
copy .env.example .env
notepad .env
```

Etape D - Charger MySQL

```
# Via MySQL Workbench (recommande)
# File > Open SQL Script... et executer dans l'ordre :
# 01_schema.sql
# 02_seed_static.sql
# 03_seed_modules.sql
# 04_seed_data.sql
```

Etape E - Lancer les 3 services

```
# Terminal 1 - academic-service
cd services\academic-service
$env:PYTHONIOENCODING="utf-8"
py -m uvicorn app.main:app --reload --port 8002

# Terminal 2 - chatbot-service
cd services\chatbot-service
$env:PYTHONIOENCODING="utf-8"
py -m uvicorn app.main:app --reload --port 8001

# Terminal 3 - frontend
cd frontend
npm start
```

13.4 Verification

URL	Resultat attendu
http://localhost:8001/docs	Swagger UI chatbot
http://localhost:8001/api/health	{"status":"ok"}
http://localhost:8002/docs	Swagger UI academic
http://localhost:8002/api/overview	JSON avec compteurs
http://localhost:4200	Dashboard FSBM

Chapitre 14 - Preparation a la soutenance

14.1 Plan de presentation (20 min)

Temps	Contenu
0-2 min	Introduction : contexte FSBM, probleme, equipe
2-4 min	Objectifs SMART et personas
4-7 min	Architecture micro-services (diagramme)
7-10 min	Demo live : chatbot multilingue (FR, EN, Darija)
10-13 min	Demo : navigation dashboard, filieres, modules
13-15 min	Architecture backend : MySQL + MongoDB + FastAPI
15-17 min	Choix techniques justifies (3-4 cles)
17-19 min	Limites + perspectives Phase 2
19-20 min	Conclusion et remerciements

14.2 Questions probables du jury

Q : Pourquoi pas un LLM comme ChatGPT pour le chatbot ?

R : Coûts (0.01-0.10 USD/requete), hallucinations (informations fausses sur la FSBM), latence (500-2000 ms vs 5 ms), explicabilite (boite noire), dependance internet. TF-IDF gratuit, deterministe, explicable, fonctionne offline.

Q : Pourquoi micro-services pour un PFE de licence ?

R : Demonstration pedagogique de patterns industriels, separation des responsabilites, stack heterogene (SQL+NoSQL), evolutivite independante. Chaque service est utilement deploye seul si besoin.

Q : Comment scaler si 10 000 utilisateurs ?

R : Plusieurs leviers : (1) repliquer chaque micro-service horizontalement derriere un load balancer, (2) cache Redis pour les requetes frequentes, (3) replication MySQL master-slave, (4) MongoDB sharding sur les logs.

Q : Comment gerez-vous la securite des donnees etudiantes ?

R : Phase 1 : validation Pydantic + anti-injection SQL + CORS strict + secrets en .env. Phase 2 : JWT avec bcrypt 12 rounds, RBAC (4 roles), rate limiting, audit logs MongoDB.

Q : Pourquoi MySQL ET MongoDB ?

R : Pratique industrielle 'polyglot persistence'. MySQL pour donnees fortement liees (etudiants, filieres) avec ACID complet. MongoDB pour donnees evolutives (reviews, logs) avec schema flexible et ecritures massives performantes.

Q : Comment avez-vous teste le projet ?

R : Tests fonctionnels : 60+ questions en FR/EN/Darija. Performance : ~150-300 ms par reponse, < 1.5 sec chargement Angular. NLP : 14/14 tests langue detectee, 11/12 puis 5/5 apres ajustements sur intents news.

Q : Quelles sont les limites de votre projet ?

R : (1) NLP TF-IDF ne generalise pas aux formulations totalement nouvelles, (2) auth JWT et 4 services en Phase 2 non implements, (3) testabilite : pas de tests automatises, (4) fsbm.ma est une SPA JS donc scraping limite, (5) pas de dashboard admin.

Q : Comment maintenir les donnees a jour ?

R : Le formulaire Google Forms genere pour le responsable scolarite permet de collecter les donnees officielles. Le faq_dataset.json est versionne et facile a etendre - ajouter un intent prend 5 minutes.

Q : Pourquoi pas de tests unitaires ?

R : Manque de temps en Phase 1, prevu pour Phase 2 avec pytest pour le backend et Karma/Jasmine pour Angular. Architecture testable (separation des responsabilites).

Q : Comment integrer ce systeme a la FSBM en vrai ?

R : Etapes : (1) deploiement sur serveur Linux univ (Apache + unicorn workers), (2) integration auth via Apogee/CAS, (3) collecte donnees reelles via le formulaire deja prepare, (4) validation contenu par responsable scolarite, (5) phase pilote 1 promo, puis generalisation.

14.3 Points techniques impressionnants a mettre en avant

- **Architecture 6 micro-services** avec 2 deja livres, separation BDD SQL/NoSQL
- **NLP multilingue** FR/EN/Darija avec detection automatique de langue
- **Personnalisation** genre + nom : adaptation 'khoya/khti', 'Monsieur/Madame'
- **3000+ donnees realistes** generees avec noms marocains authentiques
- **Recherche web live** agregeant 3 sources (base + fsbm.ma + Facebook)
- **Mode sombre** persistant avec Signals Angular 17
- **4 PDFs livres** : rapport, guide tech, guide zombies, documentation complete

- **Setup one-click** : 1 seul input (mot de passe MySQL) pour tout installer
- **Solution definitive aux zombies de ports** : netsh excludedportrange

Chapitre 15 - Analyse professionnelle

15.1 Forces du projet

Architecture moderne. Micro-services + async + multilingue + mode sombre. Demontre une veille technologique.

Pertinence sociale. Repond a un vrai besoin (information dispersee, scolarite saturee) avec un usage potentiel reel a la FSBM.

Trilingualisme. Inclut explicitement le Darija marocain - peu d'outils du genre existent.

Donnees realistes. 3000+ etudiants avec noms et CNE authentiques rendent les demos credibles.

Documentation exhaustive. 4 PDFs + plusieurs MD + README. Le projet est repris facilement par un tiers.

Bonnes pratiques. Pydantic, async, lazy loading, separation des responsabilites, .env, gitignore.

Resolution proactive de problemes. Scripts anti-zombies, fix Windows definitif, generation auto Google Forms.

15.2 Faiblesses et limites

Phase 2 incomplete. 4 services scaffoldés mais sans logique métier (student, review, notification, analytics). Ce sont des squelettes.

Pas de tests automatisés. Aucun test pytest ni Karma. Validation faite manuellement, pas reproductible facilement.

NLP limite. TF-IDF + cosine fait des choix lexicaux, pas sémantiques. Ne generalise pas a des formulations totalement inedites.

Scraping fsbm.ma limite. Le site est une SPA JS, le scraping HTTP brut renvoie peu. Necessiterait Playwright.

Pas de monitoring. Pas de Prometheus, Grafana, ou logging structure. Difficile a debugger en prod.

Authentification manquante. Pas d'auth en Phase 1 = pas de personnalisation par utilisateur reel.

Mobile non optimise. Responsive de base, mais pas d'experience native mobile (notifications push, etc.).

15.3 Scalabilite

L'architecture micro-services est **nativement scalable**. Voici les actions a mener pour passer de 100 a 10 000 utilisateurs concurrents :

Niveau 1 (1-100 users). Setup actuel suffit. Pas d'optimisation necessaire.

Niveau 2 (100-1k users). Workers uvicorn (uvicorn --workers 4), cache Redis pour overview.

Niveau 3 (1k-10k users). Repliquer chaque service x3, MySQL master-slave, MongoDB replica set.

Niveau 4 (10k+ users). Kubernetes, autoscaling, CDN pour Angular, sharding BDD.

15.4 Pertinence academique

Ce projet coche les cases attendues d'un PFE Licence DI :

- Maitrise d'un framework front moderne (Angular)
- Conception d'API REST avec validation
- Modelisation base relationnelle (16 tables 3NF)
- Introduction au NoSQL (MongoDB)
- Notions d'IA appliquee (NLP, TF-IDF, cosine)
- Architecture distribuee (micro-services)
- Gestion de projet (4 livrables PDF, scripts, doc)
- Travail d'equipe (3 membres, repartition claire)
- Documentation technique (4 PDFs detailles)
- Resolution de problemes reels (zombies WSL, encoding, etc.)

Chapitre 16 - Conclusion

16.1 Recapitulatif

Ce projet de fin d'etudes a permis de concevoir et realiser une **plateforme universitaire intelligente complete** repondant aux besoins informationnels des etudiants de la FSBM. Le resultat depasse le simple chatbot : c'est un veritable ecosysteme de services interconnectes (chatbot multilingue, referentiel academique, donnees realistes, interface moderne) bati selon les meilleures pratiques de l'industrie logicielle.

16.2 Apports techniques

- Maitrise de FastAPI + SQLAlchemy 2.0 async + Pydantic v2
- Maitrise d'Angular 17 standalone + Signals + lazy loading
- Implementation d'un moteur NLP multilingue de zero
- Conception et generation de 3000+ donnees realistes
- Architecture micro-services SQL + NoSQL
- Resolution de problemes systeme Windows (zombies WSL)
- Generation programmatique de PDFs professionnels (reportlab)

16.3 Apports humains et methodologiques

- Travail en equipe avec repartition claire (backend / frontend / contenu)
- Documentation systematique a chaque etape
- Gestion de versions et iteration (v1, v2, v3 du dataset)
- Adaptation aux contraintes Windows et choix techniques imposes
- Communication trilingue (FR / EN / Darija marocain)

16.4 Perspectives

Le projet est conçu comme une **base solide et extensible**. La Phase 2 ajoutera :

- **Authentication** JWT avec 4 roles (Student, Prof, Scholarite, Admin)
- **Reviews MongoDB** avec analyse de sentiment
- **Notifications** push en temps reel via Server-Sent Events
- **Dashboard admin** avec KPIs et top intents
- **Tests automatisés** pytest + Karma/Jasmine
- **Application mobile** Flutter avec API existante

16.5 Mot final

[OK] Vision

Au-dela de la valeur technique, ce projet illustre une vision : **l'informatique au service du service public educatif marocain**. Le chatbot et la plateforme ne sont pas des prouesses techniques pour la beaute du geste, mais des outils concrets qui peuvent ameliorer le quotidien de milliers d'etudiants de la FSBM.

Nous esperons que ce travail inspire d'autres etudiants de la FSBM et au-dela a construire des outils qui resolvent des vrais problemes pour de vrais utilisateurs - avec rigueur technique, pedagogie et ouverture sur la langue de leurs pairs.

*AKRAM BELMOUSSA, ZAKARIA, NOUHAILA
Faculte des Sciences Ben M'Sick - 2025/2026*

Glossaire

API REST. Architecture HTTP pour exposer des donnees structurees (typiquement JSON).

Apogee. Logiciel de gestion etudiante des universites marocaines.

Async/await. Modele de programmation asynchrone evitant le blocage I/O.

Bootstrap (Angular). Demarrage de l'application via bootstrapApplication().

CNE. Code National Etudiant - identifiant unique de l'etudiant marocain.

Cosine similarity. Mesure de similarite entre 2 vecteurs (0 = orthogonaux, 1 = identiques).

Dependency Injection. Pattern ou les dependances sont fournies de l'exterieur (FastAPI, Angular).

Lazy loading. Chargement d'un composant uniquement au moment ou il est necessaire.

Lifespan. Mecanisme FastAPI pour executer du code au demarrage/arret.

Micro-service. Service autonome avec une mission unique, communique via reseau.

n-gramme. Sequence de N tokens consecutifs (unigramme, bigramme, trigramme).

NLP. Natural Language Processing - traitement automatique du langage.

ORM. Object-Relational Mapping - traduit des objets en SQL et inversement.

Pydantic. Lib Python de validation par typage.

Signal (Angular). Primitive de reactivite Angular 17 (alternative a RxJS).

SPA. Single Page Application - app web qui charge un seul HTML puis manipule le DOM.

Stemming. Reduction d'un mot a sa racine ('mangeait' -> 'mang').

Stopwords. Mots tres frequents sans valeur semantique ('le', 'de', 'et').

Swagger. Specification OpenAPI generee automatiquement par FastAPI.

TF-IDF. Term Frequency - Inverse Document Frequency - mesure d'importance lexicale.